

Sri Sivasubramaniya Nadar College of Engineering, Chennai

(An Autonomous Institution affiliated to Anna University)

Machine Learning Algorithms Laboratory

Experiment 6

Bagging, Boosting, and Stacked Ensemble Models

Name	MAHALINGAM S S
Register No.	3122247001034
Degree & Branch	MTech Computer Science and Engineering(int)
Semester	V
Subject Code & Name	ICS1512 – Machine Learning Laboratory
Academic Year	2026–2027 (Even)
Batch	2024–2029

Aim and Objective

To understand and implement Bagging, Boosting and Stacked Ensemble models and compare their performance using different evaluation metrics and cross-validation.

- To understand ensemble learning strategies: Bagging, Boosting, and Stacking.
- To implement Bagging and Boosting classifiers.
- To build a Stacked Ensemble model using multiple base learners.
- To compare ensemble models in terms of accuracy, stability, and generalization.
- To analyze the effect of ensemble methods on bias and variance.

Theory

Bagging (Bootstrap Aggregation): Bagging trains multiple models on different bootstrap samples of the training data and combines their predictions. It is mainly used to reduce variance and is suitable for models such as decision trees that may have high variance.

Boosting: Boosting trains models sequentially. Each new learner gives more attention to errors made by previous learners. It is mainly used to reduce bias. AdaBoost and Gradient Boosting are common boosting methods.

Stacking: Stacking combines different types of base learners and uses a meta-learner to combine their predictions. In this experiment, SVM, Gaussian Naive Bayes and Decision Tree are used as base learners, while Logistic Regression is used as the final estimator.

Dataset Description

The Wisconsin Diagnostic Breast Cancer dataset was used in this experiment. It contains 569 samples and 30 numerical features related to characteristics of breast cell nuclei. The target variable represents benign or malignant diagnosis.

- Total samples: 569
- Numerical features: 30
- Target classes: Benign (B) and Malignant (M)

Preprocessing

The diagnosis values were converted into numerical values, with Malignant mapped to 1 and Benign mapped to 0. The ID column was removed before model training. The dataset contained no missing values. The data was divided into training and testing sets using an 80:20 stratified split, giving 455 training samples and 114 testing samples. StandardScaler was used inside the SVM pipeline.

The class distribution was 357 benign samples and 212 malignant samples.

Implementation Details

Four ensemble models were implemented:

- Bagging Classifier with Decision Tree as the base estimator.
- AdaBoost Classifier with Decision Tree as the base estimator.
- Gradient Boosting Classifier.
- Stacked Ensemble using SVM, Gaussian Naive Bayes and Decision Tree as base models, with Logistic Regression as the meta-learner.

A 5-fold StratifiedKFold cross-validation strategy was used for hyperparameter evaluation. GridSearchCV was used for Bagging, AdaBoost and Gradient Boosting.

Hyperparameter Search

For Bagging, the search considered number of estimators, maximum samples and maximum features. The best configuration obtained using accuracy scoring was 25 estimators, 0.5 maximum samples and 0.5 maximum features, with a cross-validation accuracy of 97.1429%. A separate F1-score search gave a best cross-validation F1 score of 0.9621.

For AdaBoost, the best configuration was 150 estimators, learning rate 1.0 and decision-tree maximum depth 2, giving a cross-validation accuracy of 97.5824%.

For Gradient Boosting, the best configuration was 150 estimators, learning rate 0.2 and maximum depth 2, giving a cross-validation accuracy of 97.5824%.

Table 1: Best Hyperparameter Results

Model	Best Parameters	CV Accuracy
Bagging	$n_estimators = 25$, $max_samples = 0.5$, $max_features = 0.5$	97.1429%
AdaBoost	$n_estimators = 150$, $learning_rate = 1.0$, $max_depth = 2$	97.5824%
Gradient Boosting	$n_estimators = 150$, $learning_rate = 0.2$, $max_depth = 2$	97.5824%

Performance Comparison

The final models were evaluated on the test set using accuracy, precision, recall and F1 score.

Table 2: Performance Comparison of Ensemble Models

Model	Accuracy (%)	Precision	Recall	F1 Score
Bagging	96.4912	1.0000	0.9048	0.9500
AdaBoost	97.3684	1.0000	0.9286	0.9630
Gradient Boosting	96.4912	1.0000	0.9048	0.9500
Stacking	96.4912	1.0000	0.9048	0.9500

AdaBoost obtained the highest test accuracy of 97.3684% and the highest F1 score of 0.9630 among the four models.

Confusion Matrices

The confusion matrices below are the figures generated from the submitted Assignment 6 notebook.

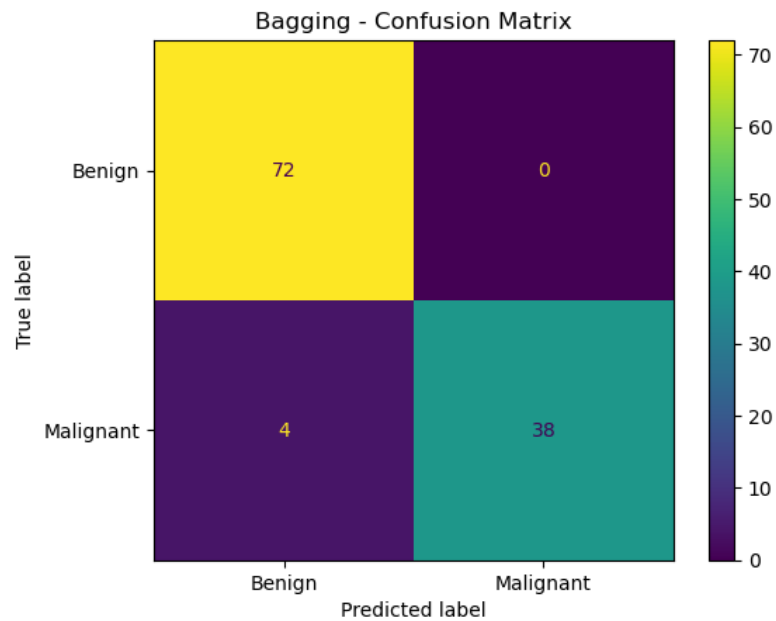


Figure 1: Bagging – Confusion Matrix

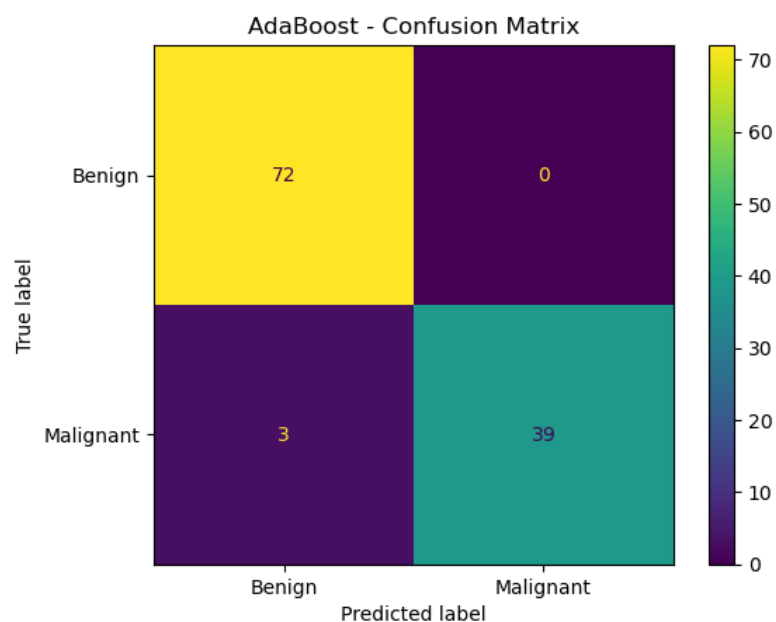


Figure 2: AdaBoost – Confusion Matrix

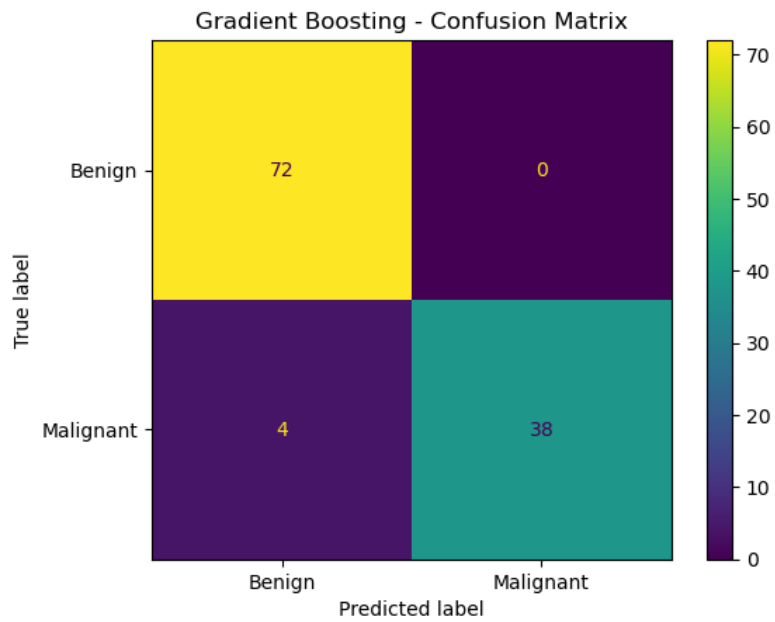


Figure 3: Gradient Boosting – Confusion Matrix

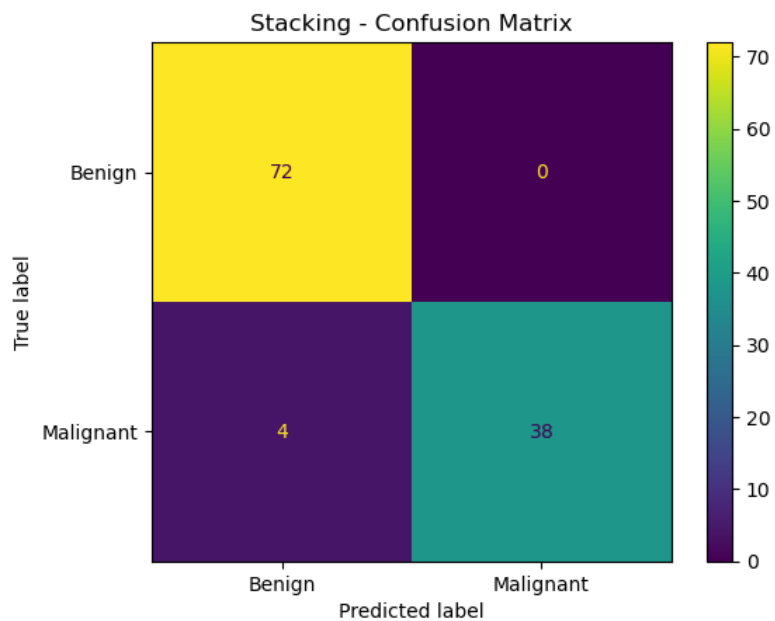


Figure 4: Stacking – Confusion Matrix

ROC Curve and AUC

The ROC curve compares the four ensemble models. The AUC values obtained in the notebook are 0.993 for Bagging, 0.989 for AdaBoost, 0.995 for Gradient Boosting and 0.994 for Stacking.

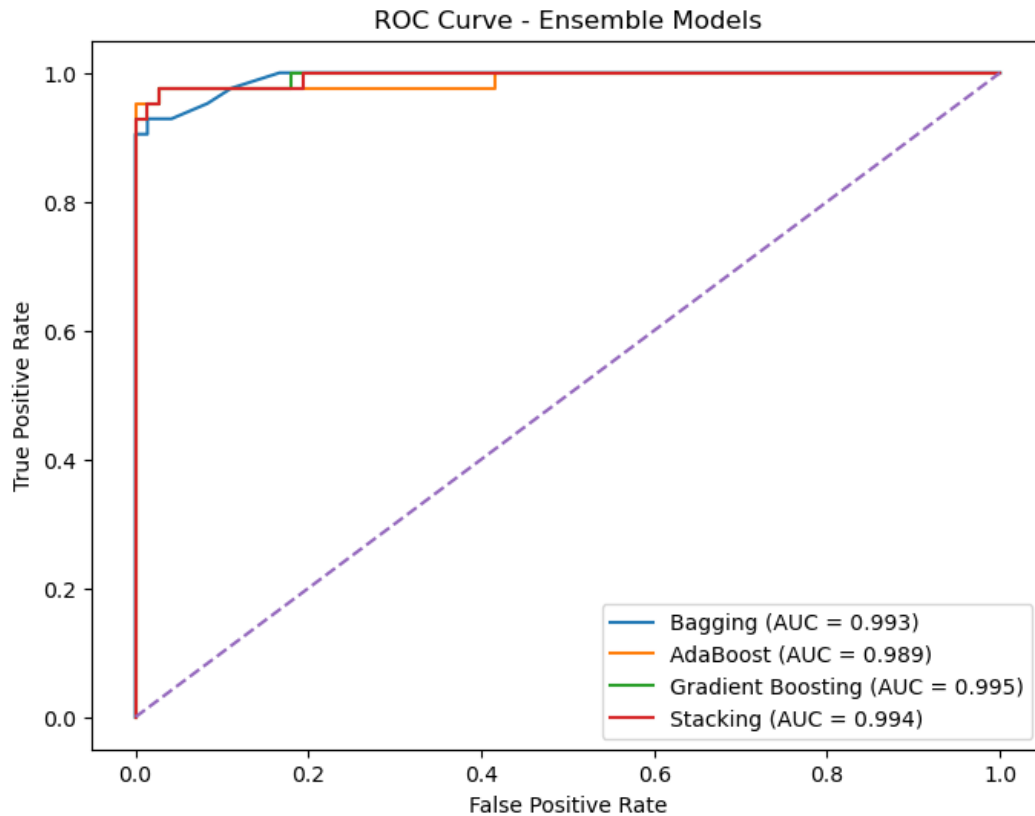


Figure 5: ROC Curve – Ensemble Models

Observations

- Bagging reduces variance by training multiple models on bootstrap samples and aggregating their predictions.
- Boosting reduces bias by training models sequentially and focusing on errors made by previous learners.
- AdaBoost gives more importance to difficult or misclassified samples.
- Gradient Boosting learns from errors or residuals of previous models.
- Stacking uses different base models and a meta-learner to combine their predictions.
- 5-fold stratified cross-validation provides a more reliable basis for hyperparameter selection.
- Among the models tested, AdaBoost produced the highest test accuracy, while Gradient Boosting produced the highest ROC-AUC.

Learning Outcomes

- Understood the working of Bagging, Boosting and Stacking.
- Learned how Bagging helps reduce variance and Boosting helps reduce bias.
- Implemented Bagging, AdaBoost, Gradient Boosting and Stacked Ensemble models.
- Used 5-fold stratified cross-validation for hyperparameter evaluation.
- Evaluated ensemble models using accuracy, precision, recall, F1 score, confusion matrices and ROC-AUC.

Conclusion

Bagging, AdaBoost, Gradient Boosting and Stacking models were implemented using the Wisconsin Diagnostic Breast Cancer dataset. Hyperparameters were evaluated using 5-fold cross-validation and the models were compared using standard classification metrics. AdaBoost achieved the highest test accuracy of 97.3684%, while all four models achieved a precision of 1.0 on the test set. The experiment demonstrates how ensemble learning can improve generalization by reducing variance, reducing bias and combining diverse learners.

Complete Python Code

The following is the complete code used in the submitted Assignment 6 notebook, kept in the same order.

```
% Code Cell 1
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV,
    StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier, AdaBoostClassifier,
    GradientBoostingClassifier
from sklearn.ensemble import StackingClassifier

from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, ConfusionMatrixDisplay,
    roc_curve, roc_auc_score
)

# Load dataset
data = pd.read_csv("wdbc.data", header=None)

# Column names
columns = ["id", "diagnosis"] + [f"feature_{i}" for i in range(1, 31)]
data.columns = columns

# Display dataset
print(data.head())
print(data.shape)

% Code Cell 2
# Convert diagnosis into numerical values
data["diagnosis"] = data["diagnosis"].map({"M": 1, "B": 0})

# Remove ID column
X = data.drop(columns=["id", "diagnosis"])
```

```

y = data["diagnosis"]

print("Missing values:")
print(X.isnull().sum().sum())

print("\nClass distribution:")
print(y.value_counts())

% Code Cell 3
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])

% Code Cell 4
cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

% Code Cell 5
bagging = BaggingClassifier(
    estimator=DecisionTreeClassifier(random_state=42),
    random_state=42
)

bagging_params = {
    "n_estimators": [10, 25, 50, 100],
    "max_samples": [0.5, 0.75, 1.0],
    "max_features": [0.5, 0.75, 1.0]
}

bagging_grid = GridSearchCV(
    bagging,
    bagging_params,
    cv=cv,
    scoring="accuracy",
    n_jobs=-1
)

bagging_grid.fit(X_train, y_train)

```

```

print("Best Bagging Parameters:")
print(bagging_grid.best_params_)

print("Best CV Accuracy:")
print(bagging_grid.best_score_)

% Code Cell 6
bagging_f1 = GridSearchCV(
    bagging,
    bagging_params,
    cv=cv,
    scoring="f1",
    n_jobs=-1
)

bagging_f1.fit(X_train, y_train)

print("Best Bagging F1 Score:")
print(bagging_f1.best_score_)

% Code Cell 7
adaboost = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(random_state=42),
    random_state=42
)

adaboost_params = {
    "n_estimators": [50, 100, 150],
    "learning_rate": [0.01, 0.1, 1.0],
    "estimator__max_depth": [1, 2, 3]
}

adaboost_grid = GridSearchCV(
    adaboost,
    adaboost_params,
    cv=cv,
    scoring="accuracy",
    n_jobs=-1
)

adaboost_grid.fit(X_train, y_train)

print("Best AdaBoost Parameters:")
print(adaboost_grid.best_params_)

print("Best AdaBoost CV Accuracy:")

```

```

print(adaboost_grid.best_score_)

% Code Cell 8
gradient = GradientBoostingClassifier(
    random_state=42
)

gradient_params = {
    "n_estimators": [50, 100, 150],
    "learning_rate": [0.01, 0.1, 0.2],
    "max_depth": [1, 2, 3]
}

gradient_grid = GridSearchCV(
    gradient,
    gradient_params,
    cv=cv,
    scoring="accuracy",
    n_jobs=-1
)

gradient_grid.fit(X_train, y_train)

print("Best Gradient Boosting Parameters:")
print(gradient_grid.best_params_)

print("Best Gradient Boosting CV Accuracy:")
print(gradient_grid.best_score_)

% Code Cell 9
svm = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", SVC(probability=True, random_state=42))
])

naive_bayes = GaussianNB()

decision_tree = DecisionTreeClassifier(
    max_depth=3,
    random_state=42
)

base_models = [
    ("svm", svm),
    ("nb", naive_bayes),
    ("dt", decision_tree)
]

```

```

stacking = StackingClassifier(
    estimators=base_models,
    final_estimator=LogisticRegression(max_iter=1000),
    cv=5
)

stacking.fit(X_train, y_train)

print("Stacking model trained successfully.")

% Code Cell 10
models = {
    "Bagging": bagging_grid.best_estimator_,
    "AdaBoost": adaboost_grid.best_estimator_,
    "Gradient Boosting": gradient_grid.best_estimator_,
    "Stacking": stacking
}

results = []

for name, model in models.items():

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    results.append([
        name,
        accuracy * 100,
        precision,
        recall,
        f1
    ])

results_df = pd.DataFrame(
    results,
    columns=[
        "Model",
        "Accuracy (%)",
        "Precision",
        "Recall",

```

```

        "F1 Score"
    ]
)

print(results_df)

% Code Cell 11
for name, model in models.items():

    y_pred = model.predict(X_test)

    cm = confusion_matrix(y_test, y_pred)

    disp = ConfusionMatrixDisplay(
        confusion_matrix=cm,
        display_labels=["Benign", "Malignant"]
    )

    disp.plot()
    plt.title(name + " - Confusion Matrix")
    plt.show()

% Code Cell 12
plt.figure(figsize=(8, 6))

for name, model in models.items():

    y_prob = model.predict_proba(X_test)[: , 1]

    fpr, tpr, _ = roc_curve(y_test, y_prob)
    auc = roc_auc_score(y_test, y_prob)

    plt.plot(
        fpr,
        tpr,
        label=f"{name} (AUC = {auc:.3f})"
    )

plt.plot([0, 1], [0, 1], linestyle="--")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Ensemble Models")
plt.legend()
plt.show()

% Code Cell 13

```

```
print("\nPerformance Comparison")
print("=" * 60)
print(results_df.to_string(index=False))

% Code Cell 14
```